

演算子と制御構文を、書ける知識に変える。

Javaのコードは「値を計算する」「条件で分ける」「同じ処理をくり返す」の組み合わせで動きます。この資料では、ゼロからでも追えるように小さな例で確認し、すぐに手を動かせる問題を一つずつ用意しました。

CONTENTS

01 演算子の考え方

02 算術演算子

03 代入とインクリメント

04 比較演算子

05 論理演算子

06 if文

07 switch文

08 for / while

09 break / continue

10 まとめ問題

01

演算子は「値を作る記号」

演算子とは、数値や文字列、真偽値を使って新しい値を作るための記号です。たとえば `+` は足し算にも、文字列の連結にも使えます。

まずは式として見る

Javaでは、値と演算子を組み合わせたものを「式」と考えると理解しやすくなります。式は評価されると、最終的に一つの値になります。

```
int price = 1200 + 300;    // 1500
String name = "Java" + "入門"; // "Java入門"
boolean ok = 10 > 3;      // true
```

大切な見方

コードを読むときは「この式の結果は何になるか」を声に出すと、急に見通しがよくなります。制御構文も、条件式の結果が `true` か `false` かで動きが決まります。

重要: 演算子を覚える目的は記号を暗記することではありません。式の結果を予測できるようになることです。

練習問題 01

次のコードを実行したとき、画面には何が表示されるでしょうか。

```
int a = 8;  
int b = 3;  
System.out.println(a + b);  
System.out.println("a + b = " + a + b);
```

▶ 解答例と説明

02

算術演算子は、計算の基本セット

足す、引く、掛ける、割る、余りを求める。プログラムの中の数値処理は、この基本セットから始まります。

演算子	意味	例	結果
+	足し算	10 + 3	13
-	引き算	10 - 3	7
*	掛け算	10 * 3	30
/	割り算	10 / 3	3
%	割った余り	10 % 3	1

重要: `int / int` の結果は小数になりません。`10 / 3` は `3.333...` ではなく `3` です。小数まで出したいときは `10.0 / 3` のように、どちらかを小数にします。

% は「周期」を扱うときに強い

余りは、偶数・奇数の判定や、何回ごとに処理するかを決める場面でよく使います。

```
int number = 14;
boolean isEven = number % 2 == 0; // true
```

計算の順番

Javaでも数学と同じように、掛け算・割り算・余りが足し算・引き算より先です。順番を明確にしたいときは、迷わず `()` を使います。

```
int result = (8 + 2) * 3; // 30
```

練習問題 02

税込価格を求めるコードを書いてください。税率は10%、商品価格は `1200` 円とします。

条件: 変数 `price` と `taxIncluded` を使い、最後に税込価格を表示します。

▶ **解答例と説明**

03

代入は「箱の中身を更新する」操作

`=` は「等しい」ではなく、右側の結果を左側の変数に入れる記号です。Java学習の最初につまずきやすい、でもとても大事な違いです。

STEP 1

右側を先に計算

`count + 1` の結果を作ります。

STEP 2

左側へ入れる

計算結果を `count` に代入します。

STEP 3

古い値は置き換わる

変数の中身は新しい値になります。

```
int count = 5;
count = count + 1; // 6
count += 1;        // 7
count++;           // 8
```

重要: `count = count + 1` は数学の式ではなく、更新処理です。「右の結果を左に入れる」と読むと、混乱しにくくなります。

練習問題 03

所持金 `money` が 1000 円あります。`drinkPrice` が 180 円の飲み物を1本買った後の所持金を表示してください。

▶ 解答例と説明

04

比較演算子は、結果が必ず `true` か `false`

比較演算子は、条件分岐の材料を作ります。「大きいか」「同じか」「違うか」を調べ、その結果として真偽値を返します。

演算子	意味	例
<code>==</code>	等しい	<code>score == 100</code>
<code>!=</code>	等しくない	<code>score != 0</code>
<code>></code>	より大きい	<code>age > 20</code>
<code>>=</code>	以上	<code>age >= 20</code>
<code><</code>	より小さい	<code>stock < 5</code>
<code><=</code>	以下	<code>stock <= 5</code>

重要: `=` は代入、`==` は比較です。「同じかどうか」を調べたいときは、必ず `==` を使います。

練習問題 04

`score` が `80` 以上なら `true`、そうでなければ `false` を表示するコードを書いてください。

▶ 解答例と説明

05

論理演算子は、条件を組み合わせる

条件が一つだけなら比較演算子で足ります。複数の条件をまとめたいときに使うのが、論理演算子です。

&& は「かつ」

両方の条件が `true` のときだけ、全体が `true` になります。

```
int age = 22;  
boolean hasTicket = true;  
  
boolean canEnter = age >= 18 && hasTicket;
```

|| は「または」

どちらか一つでも `true` なら、全体が `true` になります。

```
boolean isStudent = false;  
boolean hasCoupon = true;  
  
boolean discount = isStudent || hasCoupon;
```

! は「反対にする」

`true` を `false` に、`false` を `true` にします。

```
boolean isClosed = false;  
System.out.println(!isClosed); // true
```

重要: 複雑な条件は、読みやすい変数名に分けると一気にわかりやすくなります。条件式は短く、名前で意味を補うのが実務でも強い書き方です。

練習問題 05

年齢が 18 以上、かつ会員である場合だけ true を表示するコードを書いてください。

▶ 解答例と説明

06

if 文は、条件によって処理を選ぶ

if 文は、条件式が `true` のときだけ中の処理を実行します。 `else` を使うと、条件に合わなかった場合の処理も書けます。

```
if (条件式) {  
    条件式がtrueのときの処理  
} else {  
    条件式がfalseのときの処理  
}
```

合格判定

```
int score = 72;  
  
if (score >= 60) {  
    System.out.println("合格");  
} else {  
    System.out.println("再挑戦");  
}
```

複数条件

```
int score = 92;  
  
if (score >= 90) {  
    System.out.println("S");  
} else if (score >= 80) {  
    System.out.println("A");  
}
```

```
} else {  
    System.out.println("B");  
}
```

重要: `else if` は上から順番に判定され、最初に `true` になった場所だけ実行されます。その後の条件は見ません。

練習問題 06

購入金額 `amount` が `3000` 円以上なら "送料無料"、そうでなければ "送料500円" と表示してください。

▶ 解答例と説明

07

switch 文は、値ごとに処理を分ける

「1ならこれ、2ならこれ、3ならこれ」のように、同じ変数の値で分岐するときは `switch` が読みやすいことがあります。

```
int menu = 2;

switch (menu) {
    case 1:
        System.out.println("コーヒー");
        break;
    case 2:
        System.out.println("紅茶");
        break;
    default:
        System.out.println("水");
        break;
}
```

重要: 古い書き方の `switch` では、各 `case` の最後に `break` を書くのが基本です。書き忘れると、次の `case` まで続けて実行されることがあります。

新しい書き方もある

新しめのJavaでは、`->` を使う `switch` も書けます。この形式では、基本的に `break` を書かずに済みます。

```
int menu = 2;
```

```
switch (menu) {  
    case 1 -> System.out.println("コーヒー");  
    case 2 -> System.out.println("紅茶");  
    default -> System.out.println("水");  
}
```

練習問題 07

day が 1 なら "月曜日"、2 なら "火曜日"、それ以外なら "未登録" と表示してください。

▶ 解答例と説明

08

くり返しは、同じ処理を短く書く力

`for` は回数が見えているとき、`while` は条件が続く間くり返したいときに使うと考えると、最初は整理しやすいです。

`for` 文

```
for (int i = 1; i <= 3; i++) {  
    System.out.println(i);  
}
```

`i` を 1 から始め、`i <= 3` の間だけ実行し、1回終わるたびに `i++` します。

`while` 文

```
int count = 1;  
  
while (count <= 3) {  
    System.out.println(count);  
    count++;  
}
```

条件式が `true` の間、処理をくり返します。更新を書き忘れると、終わらないループになります。

重要: ループは「初期値」「続ける条件」「更新」の3点セットで読みます。どれかがズレると、回数が1

回多い・少ない・終わらない、というミスにつながります。

練習問題 08

`for` 文を使って、`1` から `5` までの数を1行ずつ表示してください。

▶ 解答例と説明

09

break と continue は、ループの流れを変える

`break` はループを終了し、`continue` は今回の処理だけ飛ばして次の回へ進みます。

break : 見つけたら終わる

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break;  
    }  
    System.out.println(i);  
}
```

表示は 1、2 までです。i が 3 になった時点でループを抜けます。

continue : 今回は飛ばす

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

表示は 1、2、4、5 です。3 の回だけ表示を飛ばします。

重要: `break` は「終了」、`continue` は「スキップ」です。名前だけで覚えず、処理の流れがどこへ移るかを図のように追うと定着します。

練習問題 09

1 から 10 まで表示します。ただし、偶数だけ表示してください。

ヒント: `%` と `continue` を使います。

▶ 解答例と説明

10

まとめ問題: 小さな条件判定プログラム

最後に、演算子と制御構文を組み合わせてみましょう。大きく見えても、分解すれば一つずつの部品です。

まとめ問題

テスト点数 `score` を使って、次の条件で結果を表示してください。

- 90 点以上なら "とても良い"
- 70 点以上なら "良い"
- 60 点以上なら "合格"
- それ以外なら "もう一度確認"

さらに、1 から `score` までの数字のうち、10 の倍数だけを表示してください。

▶ 解答例と説明

最後に確認したいこと

☐ 演算子は、式から一つの結果を作る。

☐ 比較演算子の結果は `true` か `false` 。

☐ `if` は条件式の結果で処理を分ける。

☐ `for` は初期値・条件・更新で回数を管理する。

☐ `%` は偶数判定や倍数判定で便利。

☐ 複雑な処理も、小さい条件と計算に分ければ読める。